

MoveQ: Parallel Acceleration for Memory-Safety Instrumentation on Modern LLVM

Donguk Kim (20201034)
UNIST
South Korea

Woohyeok Kang (20211004)
UNIST
South Korea

Abstract

Memory safety vulnerabilities, such as buffer overflows and Use-After-Free (UAF), account for nearly 70% of all security bugs. While dynamic instrumentation tools like MoveC offer deterministic protection, their sequential execution incurs prohibitive runtime overheads (2x–5x), limiting practical adoption. The Catamaran system (ISSTA '23) proposed mitigating this cost via parallelization but relied on the legacy LLVM 3.4 infrastructure, rendering it incompatible with modern environments.

In this paper, we present **MoveQ**, a modernized parallel runtime system for memory safety built on **LLVM 17**. We identify two fundamental architectural barriers in modern compilers: *opaque pointers* and *strict stack lifetime* optimizations. To overcome these, we introduce a metadata **Deep Copy** architecture that prevents Use-After-Return (UAR) errors caused by sharing stack-resident metadata with worker threads and a **Producer-Side Slab Allocator** that removes main-thread heap lock contention. Our evaluation on the PolyBench suite shows that MoveQ reduces execution time to as low as 0.68× of the optimized MoveC baseline (about 32% lower, i.e., roughly 1.47× speedup) while matching the spatial violation reports of sequential MoveC under our DOALL + wait-at-exit discipline.

1 Introduction

Software-only memory-safety instrumentation often sits on the hot path, so even small per-access costs add up. MoveC [1] enforces spatial and temporal safety by maintaining explicit metadata for pointers and validating it at every memory access. This design is precise and deterministic, but it can incur substantial overhead when checks are executed serially along the critical path.

We target MoveC for acceleration because it provides stronger and more predictable coverage than AddressSanitizer (ASan) [3], while remaining more practical than fully general alternatives such as SoftBoundCETS. Rather than replacing MoveC, our goal is to reduce its runtime cost by exploiting idle cores on modern multi-core CPUs.

Catamaran [4] showed that many MoveC-style checks can be offloaded to worker threads. However, its implementation relies on a “zero-copy” scheme that forwards pointers to stack-allocated metadata to workers. This assumption does not hold reliably on LLVM 17: opaque pointers break legacy typed-pointer assumptions in instrumentation, and aggressive stack lifetime optimizations may reuse stack slots immediately after a function returns. In our LLVM 17 port, this makes the original zero-copy design prone to Use-After-Return (UAR) failures when workers dereference metadata that has already gone out of scope.

We propose **MoveQ**, which evolves the Catamaran concept for the modern LLVM 17 ecosystem. We make the following contributions:

- We design a **Deep Copy** mechanism that snapshots metadata into a persistent heap buffer, ensuring data integrity and eliminating UAR vulnerabilities under aggressive optimization.
- We introduce a single-producer, bump-pointer **Slab Allocator** that reduces main-thread allocation overhead from about 15% to negligible levels (task dispatch still uses a mutex and `condition_variable`).
- We also apply a few system-level optimizations (e.g., enabling **Transparent Huge Pages (THP)**) to reduce TLB pressure, and we discuss the scalability limits that come from reusing the legacy MoveC runtime.

2 Background & Related Work

2.1 MoveC and Parallelism Opportunities

MoveC maintains disjoint metadata for each pointer and instruments programs with three main operations: lookup (retrieve metadata), update (modify metadata), and check (validate an access). These operations add verification on the side and do not change the program’s core data flow, which makes them a natural target for thread-level parallelism (TLP). In MoveQ, we focus on DOALL loops, where iterations are independent, so their metadata checks can be offloaded to worker threads without changing the loop semantics.

At runtime, MoveC represents pointer metadata as a compact fixed-size tuple (32 bytes) containing the base, bound, a status-node pointer, and flag bits. This uniform layout is convenient for MoveQ because metadata can be copied in bulk and stored in a slab without needing per-object headers.

2.2 LLVM 17 Challenges

Porting Catamaran from LLVM 3.4 to 17 raises two fundamental issues:

- (1) **Opaque pointers:** LLVM 17 enforces opaque pointers (`ptr`), removing explicit element type information (e.g., `struct.md*`). Under opaque pointers, LLVM associates types with individual memory operations (loads/stores, GEPs) rather than with the pointer values themselves, breaking Catamaran’s reliance on typed pointer identity for metadata tracking. Legacy instrumentation logic that relied on `getPointerElementType()` breaks. In MoveQ, the pass reads access size and alignment directly from Load/Store instructions and the `DataLayout`, and passes this to the runtime checks.
- (2) **Strict stack lifetime:** Modern compilers aggressively reuse stack slots. Sharing stack addresses with worker threads can lead to UAR, where a worker reads metadata after the main

Algorithm 1 MoveQ parallelization of a DOALL loop

```

1: for each iteration  $i$  in DOALL loop do
2:   run original loop body for iteration  $i$ 
3:    $\text{env} \leftarrow \text{pack\_env}(\text{live\_ins}, \text{metadata})$ 
4:    $\text{MOVEQ\_SPAWN}(\text{env})$ 
5: end for
6: wait ▷ synchronize all spawned checks

7: procedure WORKERLOOP
8:   while true do
9:      $\text{env} \leftarrow \text{WAIT\_FOR\_JOB}$ 
10:     $\text{md} \leftarrow \text{env.metadata\_snapshot}$ 
11:     $\text{MOVEC\_CHECK}(\text{md}, \text{env.access\_size})$ 
12:   end while
13: end procedure

```

thread has already returned. MoveQ removes this dependency by copying metadata into a heap buffer whose lifetime is controlled by the runtime.

2.3 Relation to Prior Work

ASan [3] uses shadow memory for fast detection but cannot catch all intra-object overflows or temporal bugs. SoftBoundCETS [2] provides strong spatial and temporal guarantees at higher overhead (2x–4x). In this work, we treat SoftBoundCETS purely as related work: a stable LLVM 17 port was not available, so our experiments focus on MoveC (and ASan as a reference sanitizer) rather than attempting to accelerate SoftBoundCETS itself. Parallel dynamic analyses such as ParaDyL and ShadowReplica explored offloading checks but often assume additional hardware resources. Catamaran [4] first showed that MoveC-style checks can be parallelized on LLVM 3.4; MoveQ revisits this idea on LLVM 17 with a deep-copy design compatible with opaque pointers and strict stack lifetime, while reusing the existing MoveC runtime.

3 System Design & Implementation

MoveQ consists of three main components: a task-based parallelization strategy, a Deep Copy mechanism with a producer-side Slab Allocator, and lightweight runtime optimizations for the thread pool and memory subsystem.

3.1 Overall Architecture

For each instrumented DOALL loop, MoveQ groups metadata operations for each iteration (or a small chunk of iterations) into a *task*. The main thread executes the original loop body while constructing task descriptors that hold copies of the relevant metadata. Instead of using a single global queue, the runtime maintains *per-worker job slots*: the producer assigns each task to an idle worker’s slot, and each worker thread waits on its own *condition_variable* until a job appears in its slot. A synchronization primitive (`wait()`) at loop exit ensures that all outstanding tasks complete before leaving the parallel region, preserving the logical ordering of memory accesses while moving verification off the critical path.

3.2 Deep Copy and Slab Allocator

To avoid UAR, MoveQ abandons Catamaran’s zero-copy model and uses a metadata **Deep Copy** scheme. Instead of sharing pointers to stack-resident metadata, the main thread copies the 32-byte metadata tuple into a persistent heap buffer before spawning a task. Only pointer metadata is duplicated—the application data itself remains in place—so the copying cost is bounded by the size of the metadata structure rather than the full working set.

- Worker threads access this independent copy, decoupling their lifetime from the original stack frame.
- The metadata state seen by each task is frozen at spawn time, preventing corruption from later stack reuse.

A naive Deep Copy implementation that allocates each snapshot with `malloc` incurs heavy lock contention. MoveQ introduces a single-producer **Slab Allocator** to eliminate this overhead:

- A large linear buffer (64 MB) is allocated once at startup via `malloc`.
- The producer performs lock-free allocations via a simple bump pointer; no locks or atomics are required on the hot path.
- At synchronization points (`wait()`), the bump pointer is reset to zero, achieving $O(1)$ bulk deallocation.

This design assumes a single spawning thread per region; nested parallelism or multiple producers would require per-thread slabs or atomic bump pointers, which we leave as future work.

3.3 Runtime and LLVM 17 Integration

The thread pool replaces busy-waiting (e.g., `sched_yield`) with blocking waits using condition variables. This change cuts system time from tens of seconds to below 0.1s over the full PolyBench run in our measurements. Each worker owns a dedicated job slot protected by a mutex and a condition variable. When its slot is empty, the worker sleeps instead of spinning, and the producer wakes workers by filling their slots. This matches the implementation in `runtime/ThreadPool.cpp`, which uses per-worker slots rather than a global queue.

On the compilation side, the MoveQ pass identifies DOALL candidates with standard loop analyses and replaces sequential `MoveC_Check` calls with `MoveQ_Spawn` wrappers inside selected loops. For each Load/Store, the pass recovers the access width from the instruction’s type and `DataLayout`, and passes this width to the runtime to keep bounds validation precise under opaque pointers. At loop exits, the pass inserts a call to reset the slab, reclaiming memory in bulk.

We additionally enable **Transparent Huge Pages (THP)** during experiments to mitigate dTLB thrashing caused by MoveC’s global metadata hash table. This system-level optimization benefits both MoveC and MoveQ but is particularly important for the metadata-heavy workloads we target.

Code Contributions

MoveQ reuses the MoveC metadata layout and baseline runtime, and ports Catamaran’s parallelization idea to LLVM 17. Our code contributions are:

- **LLVM 17 pass:** A new instrumentation pass targeting DOALL loops, recovering access widths under opaque pointers via

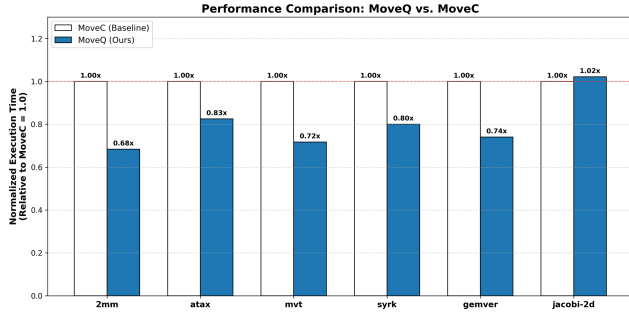


Figure 1: Normalized execution time (lower is better) of MoveQ compared to MoveC (baseline) on selected PolyBench kernels.

Load/Store and DataLayout, emitting MoveQ_Spawn, and inserting slab reset calls.

- **Deep Copy & slab allocator:** Metadata deep-copy logic plus a single-producer bump-pointer slab in the Catamaran runtime, replacing zero-copy and per-task malloc.
- **Thread pool modernization:** Replacement of sched_yield-based busy-wait with mutex + condition_variable per-worker slots, and THP-aware configuration for the metadata hash table mapping.

3.4 Correctness Considerations

MoveQ preserves the semantics of MoveC’s checks by enforcing a simple happens-before discipline:

- Only DOALL loops without loop-carried dependencies are parallelized, so each iteration can be checked independently.
- For every instrumented loop, MoveQ_Spawn is followed by a wait() at the loop exit, so all metadata checks complete (and any detected violation is raised) before the program can observe the loop’s completed effects.
- Workers operate on deep-copied metadata snapshots, so they never race with the main thread on stack-resident structures.

Together, these rules guarantee that MoveQ reports the same spatial violations as sequential MoveC, while moving the checks off the critical path.

4 Evaluation

4.1 Experimental Setup

We evaluate MoveQ on an AMD Ryzen 5 5600X (6C/12T) system running Ubuntu 22.04. We use the PolyBench/C suite with large datasets (1024^2 to 4000^2) and compile all benchmarks with -O3. The baselines are (1) a native -O3 binary and (2) the sequential MoveC instrumentation with -O3. Each configuration is executed multiple times, and we report average runtimes. We define speedup as

$$\text{speedup} = \frac{T_{\text{MoveC}}}{T_{\text{MoveQ}}}.$$

4.2 Performance Results

Speedup and Scalability. Throughout this section, figures labeled as normalized time plot $T_{\text{MoveQ}}/T_{\text{MoveC}}$ (lower is better), and the

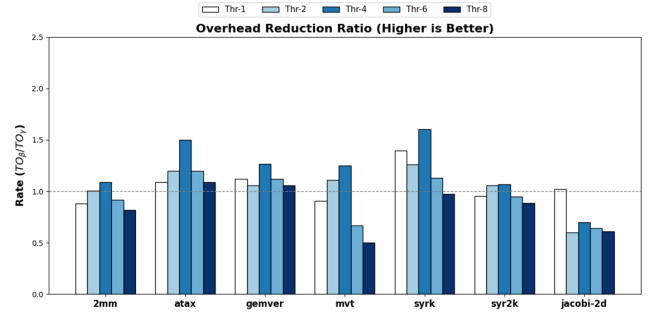


Figure 2: Overhead reduction ratio for PolyBench benchmarks. A 4-thread configuration achieves the highest average speedup.

speedup we report is its reciprocal $T_{\text{MoveC}}/T_{\text{MoveQ}}$. Figure 1 summarizes the normalized execution times of MoveQ relative to MoveC on selected kernels. MoveQ reduces runtime to $0.68\times$ (2mm), $0.83\times$ (atax), $0.72\times$ (mvt), $0.80\times$ (syrk), and $0.74\times$ (gemver), i.e., roughly 17–32% lower execution time than MoveC, while jacobi-2d shows a slight regression of $1.02\times$, where the additional Deep Copy and synchronization overhead dominates because the kernel is strongly memory-bound and offers limited slack for parallelizing metadata checks. With this speedup definition, this corresponds to about 1.2–1.5 $\times$ speedup over MoveC for the improved kernels, and the geometric mean speedup across these workloads is still above one. As shown in Figure 2, four worker threads provide the best trade-off between parallelism and synchronization overhead; beyond four threads, performance gains saturate due to lock contention in the legacy MoveC global hash table and memory bandwidth limits.

Overheads and Ablation. Deep Copy increases per-check memory usage modestly, but allocations draw from a fixed-size slab whose offset is reset at each synchronization point, keeping peak memory bounded. Replacing the slab with plain malloc reintroduces a $\sim 15\%$ slowdown on kernels such as gemm, effectively erasing most of the gains from parallelizing checks. Enabling condition variables instead of sched_yield reduces system time by several orders of magnitude, and enabling THP lowers dTLB miss rates significantly, which benefits both MoveC and MoveQ.

4.3 Functional Verification via Fault Injection

We assess correctness using **fault injection**. We synthesize code snippets that reproduce real-world CVE patterns (e.g., from libzip, libtiff, potrace) and inject out-of-bounds accesses with volatile offsets to prevent the compiler from optimizing them away. MoveQ reports the same set of spatial violations as sequential MoveC in these tests, confirming that metadata snapshots are correctly propagated to worker threads and that parallelization does not weaken spatial safety.

5 Discussion and Future Work

5.1 Limitations and Threats to Validity

MoveQ reuses the legacy MoveC runtime, including its global metadata hash table. As a result, at higher thread counts, lock contention

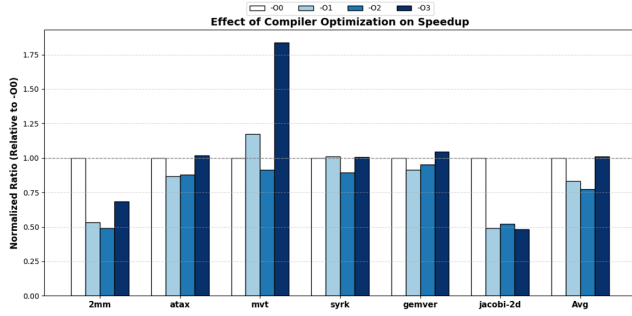


Figure 3: Effect of compiler optimization. MoveQ maintains performance gains even at the -O3 optimization level.

in the shared runtime data structures becomes a dominant bottleneck and limits scalability beyond roughly four worker threads.

Our results therefore capture two effects at once: the benefit of moving checks off the main thread and the scalability limit inherited from the legacy MoveC runtime. Deep Copy exists to make the LLVM 17 port safe: it avoids Use-After-Return by ensuring that worker threads never read stack-resident metadata after the main thread returns. That said, Deep Copy is not intended to improve temporal guarantees. Since MoveQ still uses MoveC’s status-node mechanism and snapshots are taken at spawn time, temporal behavior remains constrained by the baseline runtime semantics.

We also observed occasional temporal anomalies in 2mm. We did not diagnose every case, but the symptoms are more consistent with contention in the legacy status-node path than with MoveQ’s offloading mechanism.

Finally, our evaluation is restricted to PolyBench/C on a single Ryzen 5 5600X machine with THP enabled. Programs with heavy I/O, irregular control flow, or different memory hierarchies may expose different bottlenecks and provide less slack for offloading. PolyBench still gives a useful stress test because it includes both compute-bound and memory-bound kernels and repeatedly exercises metadata-intensive operations in MoveC/MoveQ. Our goal is not to match native -O3 performance, but to show that deterministic software-based memory safety checks can benefit from multi-core hardware when parallel slack is available.

5.2 Future Work

Future work includes:

- **Runtime modernization:** Replacing the global metadata hash table with a partitioned or lock-free structure to scale to more threads.
- **Stronger temporal safety:** Adding versioned metadata or fine-grained synchronization to reduce temporal anomalies in metadata-intensive kernels.
- **Broader language and workload support:** Extending instrumentation to C++ patterns and evaluating on server-style and mixed I/O workloads.
- **Generalizing the architecture:** Once a stable LLVM 17 port becomes available, applying the MoveQ design to other sanitizers such as SoftBoundCETS to explore the broader design space of parallel memory-safety instrumentation.

6 Conclusion

This work revisits Catamaran’s idea of offloading MoveC-style checks to worker threads and adapts it to a modern LLVM toolchain (LLVM 17). In our porting experience, the main obstacle was not the parallelization itself, but the fact that LLVM 17 no longer preserves the assumptions that the original zero-copy design relied on. In particular, opaque pointers remove typed pointer identities, and aggressive stack lifetime optimizations make it unsafe to share stack-resident metadata addresses with asynchronous workers.

MoveQ addresses this by deep-copying the per-pointer metadata tuple at spawn time, so each worker operates on a stable snapshot whose lifetime is managed by the runtime. To keep this design practical on the hot path, we also introduce a single-producer slab allocator, which avoids allocator lock contention when a large number of check tasks are created.

In our PolyBench evaluation, MoveQ reduces execution time by up to 32% relative to the optimized MoveC baseline (down to 0.68×, i.e., about 1.47× speedup). For the DOALL loops we instrument, MoveQ matches the spatial violation reports of sequential MoveC while moving the checks off the critical path.

That said, our results also make the remaining bottlenecks clear. Speedups tend to saturate beyond four worker threads, which we attribute to contention and memory-system pressure in the legacy MoveC runtime (e.g., global metadata structures). We also observe sporadic temporal anomalies in some kernels, which appear more closely tied to baseline runtime behavior under contention than to MoveQ’s task offloading mechanism itself. Overall, these findings suggest that parallelizing checks is a useful first step, but scaling further will require a more substantial modernization of the underlying shared metadata runtime.

Artifact Availability

Our implementation is available at https://github.com/zzongze0730/CSE412_Final_Project.

References

- [1] Zhe Chen et al. 2022. MoveC: High precision and deterministic memory safety enforcement. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [2] Santosh Nagarakatte, Jianzhou Zhao, Milo M.K. Martin, and Steve Zdancewic. 2012. SoftBoundCETS: A Spatial and Temporal Memory Safety Defense for C. In *ACM Transactions on Programming Languages and Systems*.
- [3] Konstantin Serebryany et al. 2012. AddressSanitizer: A fast address sanity checker. In *2012 USENIX Annual Technical Conference (USENIX ATC 12)*.
- [4] Yiyu Zhang et al. 2023. Catamaran: Low-Overhead Memory Safety Enforcement via Parallel Acceleration. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*.